

Beyond Centralized and Decentralized:

Non-Custodial Services as a Third Architectural Model

March 2025

Abstract

The prevailing taxonomy of distributed systems recognizes two architectural poles: centralized systems, in which a single authority controls both infrastructure and data; and decentralized systems, in which both infrastructure and control are distributed across independent participants. This binary has become inadequate. A third model has emerged—here termed Non-Custodial Services—in which infrastructure remains deliberately centralized for performance and reliability, but custody and control of data and identity remain exclusively with the user. This paper defines the model, situates it within a revised two-axis taxonomy, examines its technical prerequisites, and argues that its formal recognition is necessary for standards bodies, enterprise architects, and procurement frameworks grappling with identity, secrets management, and zero-trust security postures.

1. Introduction

For two decades, the architecture of internet services has been described along a single axis: centralized versus decentralized. Centralized systems—cloud platforms, identity providers, SaaS applications—offer reliability, performance, and ease of use, at the cost of entrusting sensitive data and identity to a third party. Decentralized systems—blockchain networks, federated protocols, self-hosted infrastructure—attempt to remove that trust requirement by distributing both execution and control, typically at the cost of consistency, usability, and operational complexity.

This framing has proven useful but increasingly insufficient. Practitioners in identity and access management, secrets management, and zero-trust security have long recognized a pattern that the binary cannot name: services that are architecturally centralized but in which the operator is cryptographically prevented from accessing user data. The operator runs the infrastructure. The user retains exclusive custody of their secrets. Neither party must fully trust the other.

This paper proposes a name and a framework for that pattern: Non-Custodial Services.

2. The Inadequacy of the Current Binary

2.1 What the Binary Captures

The centralized/decentralized binary captures a genuine and important dimension: the distribution of computational infrastructure. A centralized system executes on infrastructure owned and operated by a single entity. A decentralized system distributes that execution across many independent nodes. This axis has meaningful implications for fault tolerance, censorship resistance, and regulatory jurisdiction.

2.2 What It Misses

The binary collapses a second, independently variable dimension: the distribution of control over data, identity, and secrets. It assumes that whoever operates the infrastructure necessarily has access to what runs on it. For most of computing history, this assumption was technically unavoidable. It is no longer.

Advances in confidential computing—hardware-enforced trusted execution environments (TEEs) such as AWS Nitro Enclaves, Intel TDX, and AMD SEV—make it possible to operate services on centralized infrastructure in which the operator is

cryptographically excluded from the content being processed. The operator can verify that the correct code is running; they cannot observe or modify the data flowing through it.

The binary assumes that whoever operates the infrastructure necessarily controls what runs on it. Confidential computing has broken that assumption.

When custody can be separated from hosting, the binary produces a false equivalence: it treats centralized-but-non-custodial services as indistinguishable from centralized-and-custodial ones. They are not the same. The security properties, trust models, regulatory implications, and user rights they confer are fundamentally different.

3. A Revised Two-Axis Taxonomy

Separating infrastructure distribution from custody distribution yields a two-by-two taxonomy with four quadrants:

	Custodial (platform controls data)	Non-Custodial (user controls data)
Centralized Infrastructure	Traditional Cloud AWS, Google, Okta, Azure Platform runs it. Platform owns it.	Non-Custodial Services ZKT / VettID Platform runs it. You own it.
Decentralized Infrastructure	Custodial Decentralized Some DeFi Protocols Distributed runs it. Smart contracts own it.	Blockchain Identity Self-Hosted Distributed runs it. You own it.

3.1 Custodial Centralized (Traditional Cloud)

The dominant model today. A single operator controls both infrastructure and data. Examples include major cloud identity providers, SaaS platforms, and password managers that hold user vaults in operator-controlled storage. The operator is trusted by assumption, and a breach of the operator results in a breach of user data.

3.2 Non-Custodial Decentralized (Blockchain Identity / Self-Hosted)

Self-sovereign identity systems, blockchain-based credential networks, and self-hosted infrastructure distribute both execution and control. The user retains custody by virtue of operating their own nodes or controlling private keys on-chain. Resilient to operator compromise by design, but at significant cost to usability, consistency, and operational burden.

3.3 Custodial Decentralized (An Underappreciated Failure Mode)

Systems whose execution is decentralized but whose effective control remains with a small set of smart contract deployers, DAO multisig keyholders, or protocol administrators. Decentralized in appearance; custodial in practice. The decentralization of infrastructure does not guarantee the decentralization of control.

The key insight: decentralization of infrastructure and decentralization of control are independently variable. Conflating them has produced both misplaced trust in “decentralized” systems and unwarranted suspicion of

centralized ones.

3.4 Non-Custodial Services (The Emerging Model)

Infrastructure is centralized—operated by a known service provider on reliable, auditable cloud infrastructure. Custody of data, identity, and secrets remains exclusively with the user, enforced by hardware attestation and cryptographic guarantees rather than by policy or contractual obligation. The operator cannot access user secrets even if compelled, compromised, or curious.

4. Technical Prerequisites

Non-Custodial Services are not merely a policy choice. They require a specific technical substrate to be meaningful:

Hardware-enforced isolation. Trusted Execution Environments (TEEs) provide cryptographically attested compute boundaries that the operator cannot penetrate.

Remote attestation. Cryptographic verification that the correct, unmodified code is running inside the enclave before any secrets are transmitted to it.

User-controlled key derivation. User-held keys derived independently of the service operator, so that operator compromise does not yield user secrets.

Ephemeral plaintext. Protocols that prevent the service from accumulating long-lived copies of plaintext user data outside the enclave.

Auditable execution. Public, auditable code running inside the enclave, so that users can verify what they are trusting.

When these prerequisites are met, the service provider’s role becomes analogous to that of a blind trustee in financial law: they manage the infrastructure and ensure the service runs correctly, but they are structurally excluded from visibility into what the service holds on behalf of users.

The operator’s role becomes that of a blind trustee: they keep the lights on, but they cannot see inside.

5. Zero-Knowledge Trust and VettID as a Reference Implementation

Zero-Knowledge Trust (ZKT) is a security model in which a service is architected so that the operator has zero knowledge of user secrets by design, not by policy. It extends the zero-knowledge principle beyond cryptographic proofs to encompass the entire service architecture: network paths, storage, key management, and access control.

VettID is an identity and secrets management platform built on ZKT principles, using AWS Nitro Enclaves as its confidential computing substrate. It exemplifies the Non-Custodial Services model:

- Secrets are encrypted client-side before transmission and are only decrypted inside attested Nitro Enclaves.
- The VettID operator cannot access user credentials, identity assertions, or stored secrets.
- Users authenticate with multi-factor credentials whose derivation is independent of the operator’s key material.

- Credential backup and restore flows preserve the non-custodial guarantee even across device loss scenarios.

VettID’s companion protocol, LEASH (Lightweight Encrypted Agent Secret Handling), extends the Non-Custodial Services model to AI agent workflows, enabling autonomous agents to consume secrets without exposing those secrets to the AI model host or orchestration layer.

6. Implications

6.1 For Identity and Access Management

Current identity standards—SAML, OIDC, FIDO2—presuppose that the identity provider has access to the assertions it issues. Non-Custodial Services suggest a new category of identity architecture in which the provider attests to identity without ever holding the underlying credential. Standards bodies including FIDO Alliance, NIST NCCoE, and CISA should consider whether existing frameworks require extension to address this model.

6.2 For Procurement and Regulation

Regulatory frameworks such as GDPR, HIPAA, and emerging AI governance regimes impose obligations on entities that process personal data. Non-Custodial Services architecturally eliminate the operator’s ability to process user data in the plain, which has meaningful—and currently unaddressed—implications for how these obligations are assigned. A service provider who cryptographically cannot access user data occupies a different legal and liability position than one who chooses not to.

6.3 For Enterprise Security Architecture

Zero-trust network architectures ask organizations to “never trust, always verify.” Non-Custodial Services extend this principle upward to the service layer: the operator of the service is itself subject to verification rather than implicit trust. This represents a meaningful maturation of zero-trust from a network perimeter model to a full-stack trust model.

6.4 For the Decentralization Debate

Much of the appeal of blockchain-based identity and decentralized systems derives not from a desire for distributed infrastructure per se, but from a desire to escape custodial relationships with service operators. Non-Custodial Services demonstrate that this goal is achievable without sacrificing the reliability, performance, and usability of centralized infrastructure. The decentralization of control does not require the decentralization of execution.

7. Conclusion

The centralized/decentralized binary has served the field well, but it conflates two independently variable dimensions: the distribution of infrastructure and the distribution of control. As confidential computing matures and becomes operationally accessible, services that separate these two dimensions are no longer theoretical—they are in production.

Non-Custodial Services constitute a coherent, technically grounded architectural category that existing taxonomies cannot accurately describe. Naming and formalizing this category is not merely an academic exercise. It is a prerequisite for standards that govern them, regulations that address them, and enterprise architectures that evaluate them.

The question for the field is not whether to choose centralized or decentralized infrastructure. It is whether to accept custodial relationships with service operators as the cost of

using centralized infrastructure—or to demand something better.

Centralized infrastructure. Decentralized control. This is not a compromise.
It is a third way.